

Be Aware of Metadata Corruption in Parallel File System: It Can Be Silent and Catastrophic

Saisha Kamat*, Mai Zheng†, Bo Fang‡, Dong Dai§

*Computer Science Department, University of North Carolina at Charlotte, skamat1@charlotte.edu

†Department of Electrical and Computer Engineering, Iowa State University, mai@iastate.edu

‡Pacific Northwest National Laboratory, bo.fang@pnl.gov

§Department of Computer and Information Sciences, University of Delaware, dai@udel.edu

Abstract—High-Performance Computing (HPC) systems rely on parallel file systems (PFS) like Lustre to reliably manage large-scale data. Unfortunately, such data stored in PFS are subjected to corruption due to hardware failures, power outages, on-disk corruption, in-memory corruption, and software bugs. Previous work has shown the impacts of general *data* corruptions on scientific applications, but barely investigated the impacts of corruptions on *metadata*, which is a special type of data for many critical PFS functionalities. Since the metadata are accessed and updated frequently, the metadata corruption is non-negligible, and the impact on both HPC systems and applications is often complicated.

In this study, we systematically studied the effects of PFS metadata corruption on representative scientific applications and workflows through fault injections. We observed various abnormal behaviors of applications against metadata corruptions, such as failing to finish execution or successfully finishing execution but generating wrong outputs silently. We also showed that neither the defensive programming practice from the application developers nor existing metadata checking mechanism deployed in PFS can resolve the metadata corruption issues or guild applications to react correctly.

To address this issue, we further propose a metadata checksum mechanism as a system-level mitigation strategy to detect and handle metadata corruptions at runtime. We implement a prototype of the checksum mechanism on Lustre file system using FUSE interface and evaluate its functionality and performance overhead. Our results show the proposed checksum mechanism can effectively detect metadata corruptions during data accesses with extremely low overhead and stop the applications from continuous execution and generating wrong results.

I. INTRODUCTION

In High-Performance Computing (HPC) systems, the parallel file systems (PFS), such as Lustre [6], BeeGFS [29], or PVFS [19], serve an essential role of offering efficient and reliable data accessing to critical scientific applications [11, 21, 30, 31]. Although data access speed often draws more attention to achieve faster application executions, data access reliability, i.e., the ability to read/write data correctly from/to parallel file system, is also important for getting the correct scientific results, especially given the fact that parallel file systems may experience various corruptions and failures due to software bugs, hardware faults, and power outages as many previous work has shown [10, 40].

Based on the consequences, failures in parallel file systems (PFS) can be categorized into two classes: *fail-stop* and *partial failures* [39]. Fail-stop failures make the PFS unmountable or inaccessible to applications. These failures are typically highly noticeable and demand immediate attention from users and system administrators. In contrast, partial failures compromise specific PFS components and affect only a small subset of PFS operations, often allowing the system to remain functional but potentially causing unnoticed damage or silent errors to the applications, sometimes even catastrophic. For instance, they can lead to applications reading or writing corrupted data without detection (silently), and hence result in incorrect scientific results and conclusions.

Due to the severity of parallel file system failures, there have been a number of studies focusing on this [22, 33, 45, 46]. For example, PFault [18] performed fault injections to PFSes and analyzed their failure handling mechanisms. Dai et. al. [20] studied the performance and functionalities of existing PFS fault handling mechanisms. However, these studies mostly focus on the *fail-stop* cases where the PFS experiences significant errors and cannot serve applications and ignored the also critical *partial failure* cases.

On the other hand, most studies on partial failures focus on device-level partial failures [14–16, 44, 47]. Only limited studies discussed such issues in the HPC environment. For example, Narayan et. al. [35] discusses various PFS corruptions, including misdirected writes, torn writes, and bit-rot, leading to potential application errors or system failures. Fang et. al. [23] further studied the impacts of various storage faults on HPC applications. As described, these studies focus on general *data corruptions*, such as bitflip, shorn writes, and dropped writes to generic places of the storage devices, impacting the correctness of PFS data. However, they commonly overlook the impacts of corruption on the metadata of the file system, which is essentially a special type of data stored in special locations to support various PFS functionalities. Since metadata represent critical information and they are accessed frequently, their *partial failures* could potentially lead to very complicated consequences to the file system itself and the applications atop [25, 27, 34], requiring careful study to identify and address potential problems.

In this paper, for the first time, we systematically investigate the impacts of metadata corruption in the parallel file

Corresponding Authors: Dong Dai and Bo Fang. Part of this work was conducted while Dong Dai was with the University of North Carolina at Charlotte.

system on representative HPC applications or workflows. Specifically, we corrupt the PFS metadata and then assess the behavior of three applications: Nyx [11], Montage [31], and 1000Genome [30]. Through extensive fault injection experiments, we observe complicated behaviors of applications under different metadata corruption scenarios, ranging from failing to finish execution to successfully finishing execution but generating wrong outputs silently, which can lead to catastrophic consequences. We further show that neither defensive programming practice from the application developers nor existing metadata checking mechanism deployed in PFS, such as LFSC for Lustre [3], can resolve these metadata corruptions or guild applications to react correctly.

To address such issues particularly for mission-critical scientific applications, we further propose a generic checksum-based solution to accurately identify metadata corruptions and notify the applications at runtime. Through multiple evaluations, we show the new mechanisms can identify faults and alert the applications successfully. We benchmark the overheads of our prototype and show the checksum mechanism introduces unnoticeable overheads when implemented via the FUSE interface [5]. We believe directly integrating the checksum mechanisms into specific parallel file system will likely experience similar level of overhead, making checksum method a practical solution. The main contributions of this research are three-fold:

- To the best of our knowledge, this is the first study that systematically analyzes how PFS metadata corruptions impact application behaviors.
- We identify various unique application behaviors, particularly the silent cases where applications complete successfully but generate incorrect results.
- We propose a checksum-based mechanism that effectively detects metadata corruption at runtime with minimal overhead.

The rest of this paper is organized as follows. Section II first provides an overview of Lustre file system metadata and the scientific applications used in our study. In Section III, we discuss our methods to inject metadata failures and present the results on how metadata corruptions affect various applications, both with and without the use of parallel file system checking (i.e., LFSC). We then introduce the proposed checksum tool in Section IV. Section V shows the evaluation results of the naive checksum tool, including its performance and effectiveness. Section VI discusses related work, and Section VII concludes the study and discusses the future work.

II. BACKGROUND

Similar to data corruptions, the metadata corruptions can happen in any parallel file system. In this study, for facilitating detailed discussions, we focus on one of the most popular parallel file systems, Lustre [6]. As of today, Lustre has been deployed on over 75% of the Top500 supercomputers [9]. Since metadata governs the storage and retrieval of file and directory locations in various parallel file systems, the

behaviors observed in this study would occur in other parallel file systems as well.

In this section, we first provide an overview of the Lustre file system architecture and its key metadata. Next, we introduce the scientific applications used in this study: Nyx, Montage, and 1000Genome.

A. Overview of Lustre Architecture and Metadata

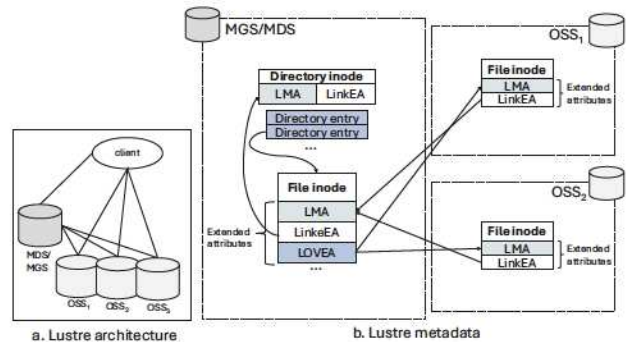


Fig. 1: Lustre Architecture and Metadata

Fig. 1.(a) illustrates a typical Lustre cluster, which includes Management Servers (MGS), Metadata Servers (MDS), and Object Storage Servers (OSS). The MGS and MDS servers are often combined and are responsible for storing configuration-related information and metadata of the file system, while the OSS servers store the actual file data. Files are striped into fixed-size chunks and stored as data objects on OSS servers. These servers often use Ext4-based local *ldiskfs* file system to store data and metadata.

As shown in Fig. 1.(b), Lustre operates by distributing metadata across MDS and OSS. Each Lustre file, directory, and data object corresponds to a file in the local *ldiskfs* file system. All the Lustre metadata are embedded in the Extended Attribute (EA) field of the inode of a corresponding local *ldiskfs* file.

To summarize, we can categorize important metadata in Lustre into two classes: 1) namespace metadata, which maintains the directory structure of the Lustre file system, such as directories and files, and 2) data layout metadata, which manages the relationships between a Lustre file and its stripes distributed across different OSS servers.

The *namespace metadata* is stored entirely on the MDS server. It involves three key parties: directory entry stored in the directory structure, and LMA, and LinkEA stored on both directory and file inodes. Here, Lustre directory entry is similar to its local file system version for storing both the local inode ID and Lustre LMA to reference child files or directories. LMA is a 128-bit file/directory identifier that holds a globally unique value for each object in Lustre. The LinkEA field stored on file/directory inode is to point back to its parent directory by storing the parent's LMA.

The *data layout metadata* is stored on both MDS and OSS servers. It mainly involves three parties: LOVEA on the file inode, LMA on both file and OSS object inodes, and LinkEA

Application	Domain	Objective
Nyx-halofinder	Astrophysics	Adaptive Mesh Refinement (AMR)-based cosmological simulation
Montage-m101	Astronomy	Astronomical image mosaic
1000Genome-mutation overlap	Genomics	Chromosome mutation overlap analysis

TABLE I: Overview of scientific applications analyzed in this study.

on the OSS object inodes. Here, the LOVEA field references all its striped OSS objects on OSS servers by pointing to their LMAs. The LinkEA of the OSS object inode points back to the file that the OSS object belongs to.

These metadata fields are critical for Lustre data access. For instance, to locate a file offset to read, Lustre needs to iterate through the directory entries level by level, find the local file inode with correct LOVEA metadata, and finally get the corresponding OSS object. It is not surprising that corruptions on such critical metadata fields could lead to problems in the applications. However, if these problems are observable, controllable, and fixable by existing storage infrastructure is not clear, which motivates this study.

B. Overview of the Scientific Applications

In this study, we conducted our experiments on three representative real-world HPC applications: Nyx [11], Montage [31], and 1000Genome [30], summarized in Table I. The common characteristic of these three applications is the high volume of I/O operations, coupled with their post-analysis processes. A higher number of I/O operations increases the likelihood of being impacted by metadata corruptions, while the post-analysis process reveals how these faults might affect applications.

1) *Nyx-halofinder workflow*: Nyx is an adaptive mesh hydrodynamics algorithm designed to model astrophysical reacting flows. It simulates dark matter as discrete particles influenced by gravity and uses a finite-volume approach on an adaptive 3D Eulerian mesh to model fluid dynamics. This mesh structure evolves both fluid quantities and particles through a particle-mesh method. Nyx includes various post-analysis programs, such as power spectrum analysis, which describes the distribution of the Universe at different scales, and dark matter halo identification. Among these, the Nyx-halofinder is a widely used tool that identifies halos based on the “baryon density” field. It processes a single input file called “NVB_C009_110n512_S12345T692_z5” from the Nyx dataset [37] and outputs “NVB_integral_128” that shows the ID, positions, cell counts, and masses of each detected halo (see Fig. 2.a for an example).

2) *Montage-m101 workflow*: Montage is a toolkit designed for creating custom mosaics from Flexible Image Transport System (FITS) images. In our study, we utilize Montage’s m101 workflow. This workflow assembles a mosaic from ten 2MASS Atlas images, covering a 0.2-degree area around m101 in the J band, and reprojects them into the TAN projection. Two versions of the mosaic are produced: one background-matched and one uncorrected. The final

Corruption type	Description	Possible corruption cause
missing	metadata field is missing or deleted	disk failure; software bugs; file system crash; incomplete writes.
random	metadata is replaced with a random value	disk bit-flips; software bugs; corrupted writes.
swap_similar	metadata is swapped with a similar file’s metadata	misdirected writes; faulty recovery mechanisms.
swap_random	metadata is swapped with metadata of a random file	misdirected writes; faulty recovery mechanisms.

TABLE II: The metadata fault model used in this study.

output includes mosaics of m101 and corresponding area images, which Montage uses to co-add the images into a single mosaic (illustrated in Fig. 3.a).

3) *1000Genome-mutation overlap workflow*: The 1000 Genomes Project offers a comprehensive reference for human genetic variation by reconstructing the genomes of 2,504 individuals from 26 global populations. In our study, we use the mutation overlap workflow, which utilizes data from the 1000 Genomes Project to identify mutational overlaps. This workflow processes Phase 3 data for ten chromosomes (chr1 to chr10) from various populations: African (AFR), Mixed American (AMR), East Asian (EAS), European (EUR), British from England and Scotland (GBR), and South Asian (SAS). It also includes the dataset for ALL populations, which aggregates data from all individuals in the latest release. The workflow cross-matches the extracted data (which person has which mutations) with the mutation’s sift score (how bad it is). Then it performs a few analyses.

In the next section, we analyze how Lustre metadata corruptions impact the behavior of the three applications.

III. APPLICATION BEHAVIORS ANALYSES

In this section, we first discuss our fault model for metadata fault injection. Then, we run the three scientific applications and analyze their behaviors. We further explain why these behaviors were generated from the source code level. We run file system checker (i.e., LFSSCK) on the corrupted file system to evaluate if existing file system (Lustre) checking mechanisms can help address the partial failure issues.

A. Fault Injection to File System Metadata

To study the behaviors of applications facing metadata corruptions, we randomly select an input file from the application’s input dataset, locate its inode on the disk, then use `setfattr()` to modify the specific metadata of the selected file, which is typically stored in the extended attribute field of the inode data structure.

In terms of the specific corruptions introduced, we use a fault model based on the literature [17, 18, 24], which discussed various metadata corruptions and issues in HPC parallel file systems. Specifically, as shown in Table II, the fault model comprises four types of corruption:

- **missing**: the corresponding metadata field is completely missing or deleted in the local inode.
- **random**: the corresponding metadata is replaced with a random value that might not make sense.

Behaviour type	Application execution	Application output	File accessible?	File access outcome
stall	application stalls indefinitely, unable to complete execution	no output	no	file access leads to stalling
stop	application stops abruptly, terminates without completing	no output	no	file access triggers application stop
detected	application detects and reports corruption, completes with partial data	wrong output	depends (on corruption)	file is either inaccessible; or accessible with wrong data or empty
silent	application runs normally	wrong output	yes	file is accessible with wrong data
correct output	application runs normally, completes with correct dataset	correct output	yes	file accessed normally-no impact from corruption

TABLE III: The definitions of different application behaviours (*symptoms*).

- **swap_similar**: the corresponding metadata is swapped with the metadata of another file in the file system. The another file may be similar to the corrupted file (generated in the previous experiments).
- **swap_rand**: the corresponding metadata is swapped with the metadata of a pre-created file, whose content is just random values.

Note that these four types of corruptions are derived from representative failure scenarios in the real world [17, 18, 24], which may affect PFS in different ways. For instance, we introduced the corruptions of `swap_similar` and `swap_rand` to simulate the scenarios where the target metadata having a value that is the same as that of another file's corresponding metadata. The another file could be either meaningful or meaningless to the application. This type of fault usually occurs while doing file-level backup and restore [38].

Besides the fault model, we need to identify which metadata fields to be corrupted. In this study, we focus only on the Lustre level metadata, such as LMA, LinKEA, and LOVEA. The metadata corruptions on the local file system might co-exist, but are not focus of this study as they have been studied in previous work [26, 32, 41]. These three metadata fields represent general metadata that widely exists in different parallel file systems: LMA managing global identifier of a data object, LinKEA managing directory structure, LOVEA pointing the location of actual data chunks.

Among these three, we further exclude LinKEA. To read or write to a file, Lustre first locates inode using the LMA metadata. Next, it accesses the file's data objects using the LOVEA metadata. For these reasons, both metadata are critical to data accesses and subject to failure analysis. In contrast, the LinKEA field is primarily used for cross-checking the correctness of directory entries and has minimal impact on file access. We confirmed this by injecting corruptions into the LinKEA field and running various applications, which continued to function without any issues. Additionally, we observed that Lustre's file system checking mechanism can detect and correctly fix LinKEA metadata corruptions. Because of these, our discussion will focus exclusively on LMA and LOVEA fields.

Note that, for selecting which file to inject faults, we do it with the consideration of the semantic of the application. In the case of Nyx, it uses a single input file, so we

target this file only. In the case of Montage, we corrupt the metadata of one of the multiple input image files that the application uses to form the mosaic. For 1000Genome, we select the file for *chromosome-1* within ALL population category (ALL.chr1.250000.vcf).

B. Classification of Application Behavior

In our experiments, we start by running the applications without fault injection and record their outputs. Next, we inject faults based on the fault model, one at a time, and rerun the application from a clean state. We then observe how they affect the file's access by the application and its impact on the application's execution and outcome. We compare this output to the original to determine the impact of the corruption. For these applications to successfully complete, their outputs must match the originals to be considered correct; otherwise, we note the differences. For instance, Fig. 2.a and Fig. 3.a show the original outputs of Nyx and Montage, where no corruption was injected. Fig. 2.b and Fig. 3.b show the outputs after a specific fault injection, LOVEA `swap_similar`, in this case. Comparing the two, we see that they do not match due to the fault injection.

For all the corruption cases, we further run the file system checker (LFCK), which is designed to fix metadata inconsistencies in Lustre, to repair the corruptions. Next, we rerun the application a third time to determine whether LFCK can mitigate the effects of metadata corruptions.

Before analyzing how applications behave under different metadata corruptions in our evaluation, we first formally define these behaviors, as shown in Table III.

- **stall**: The application stalls indefinitely, indicating that the metadata corruption is holding the file access. As a result, the application is unable to complete its execution and is waiting forever.
- **stop**: The data accesses return error due to metadata corruptions and make the application completely stop before completing execution and throw an error.
- **detected**: The application completes the execution. But it identifies and reports a problem due to the corrupted file. It then skips the file and continues processing the remaining data. As a result the output does not match the expected outcome. This can occur in two scenarios: first, metadata corruption makes the corrupted file inaccessible, so the application reports it to the user and processes the remaining files; second, the file

Halo_ID	Position	#of cell	Total_mass
8686203	123 69 33	61	30581
7630447	111 55 29	14	1825.41
14956774	230 28 57	50	14611.3
13382365	221 25 51	67	16813.4
13113049	217 11 50	63	25755.1

a. Correct output

Halo_ID	Position	#of cell	Total_mass
8423547	123 68 32	63	36393.2
16253010	82 0 62	138	45667.1
7630960	112 56 29	18	2760.61
25434836	212 13 97	27	2993.42
17052912	240 26 65	12	1194.29

b. Wrong output

Fig. 2: Fault impact on Nyx-halo finder

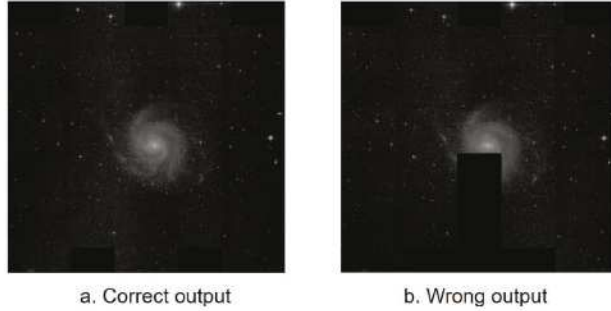


Fig. 3: Fault impact on Montage-m101

remains accessible, but the corruption alters the file format or renders it empty. In this case, the application either performs explicit validation checks and identifies file to be a "bad file" (i.e., the file does not match the expected format or is empty) or it encounters an error while processing due to the unexpected format and reports it before continuing. In both scenarios, the application does not stop but processes the remaining input files, leading to an incomplete output.

- **silent:** Application completes the execution without reporting any error, but the output does not match the expected results. This can occur due to two scenarios: first, metadata corruption affects the file access or the file format, but the application fails to report; second, the metadata corruptions does not hinder file access, nor does the application detect any issue with the file's format. The application runs as normal but produces wrong outputs as the accessed files are not correct.
- **correct output:** The application processes the entire dataset, including the file with corrupted metadata, and produces the correct results. This indicates that the metadata corruption does not affect the application; the application is accessing the correct file data and producing the correct output.

C. Key Observations and Analysis

Intuitively, it is expected that the metadata corruptions, especially those applied to the key Lustre metadata fields, would likely impact the data accesses and hence lead to obvious errors and failures in the applications. However, the actual behaviors of applications are complicated.

We present our key results in Table IV. At a high level, we observe that applications exhibit significantly different behaviors in response to various metadata corruptions. Furthermore, different applications respond differently to the same metadata corruption. Notably, the results also include

cases where we rerun the application after executing LFSCK to repair the metadata corruptions.

In this subsection, we first analyze cases without running the file system checker (LFSCK). We focus on two types of metadata corruptions: those affecting LMA and those affecting LOVEA. We describe each type separately below. Notably, we omitted results related to LinkEA metadata corruptions from this discussion. As mentioned earlier, we observed that such corruptions did not affect application execution. Additionally, we observed that LFSCK can accurately detect and repair various LinkEA corruptions.

1. LMA Corruptions Cases. Overall, we can observe that LMA corruptions mostly lead to applications to stall, meaning the application is blocked during execution. Such an issue is expected and easily observable to users at runtime. The worst consequences would just be wasting the computation resources. To be more specific, three metadata corruptions on LMA (random, swap_similar and swap_rand) would lead to such a consequence across all three applications. We believe this happens because of the Lustre OI scrub (Object Index scrub) mechanism [4]. OI scrub is an internal Lustre component that is responsible for maintaining the integrity of the OI table (Object Index table) - a table used by Lustre during file access and maps file LMA to its corresponding inode number. Our corruptions in LMA like random, swap_similar and swap_rand cause inconsistency to the OI table's integrity. As a result, OI scrub attempts to make the OI table consistent but fails, resulting in repeated retries and an indefinite stall in the application.

However, it is interesting to observe that LMA missing case will lead to different results. When LMA is missing, all the applications simply run as normal and generate correct results. This rather unexpected behavior is relevant with how Lustre handles LMA metadata. In this case, the corrupted file lacks the LMA attribute (as it is missing) and is not recognized as a Lustre object. As a result, the file access process switches from using Lustre metadata to using local *ldisks*-level metadata, such as file name and inode number, which is intact. So that the applications access data normally and generate correct results.

2. LOVEA Corruptions Cases. Comparing with LMA corruptions, LOVEA corruptions consequences are more complicated, showing varying application behaviors.

1) The LOVEA missing case leads to stop for Nyx, detected for Montage, and silent for 1000Genome. It is interesting to observe that same corruption causes different reactions in applications. Note that, LOVEA missing simply means the file exists but its OSS objects can not be located, so that the file is still accessible but its content is empty. When such a corruption occurs, Nyx stops the execution when it encounters the empty file, as the file does not meet the expected format. Montage, like Nyx, detects that the empty file does not meet the expected format. However, it does not stop; instead, it reports the file as a "badfits" (bad .fits file) and continues processing the remaining input files. In contrast, 1000Genome does not verify whether the input files match an expected format. It processes the empty file

Metadata field	Fault model	Application behavior due to Fault injection			Application behavior after LFSCCK run		
		Nyx	Montage	1000Genome	Nyx	Montage	1000Genome
LMA	missing	correct output	correct output	correct output	stop	detected	silent
	random	stall	stall	stall	correct output	correct output	correct output
	swap_similar	stall	stall	stall	silent	silent	silent
	swap_rand	stall	stall	stall	stop	detected	detected
LOVEA	missing	stop	detected	silent	stop	detected	silent
	random	stop	silent	detected	stop	silent	detected
	swap_similar	silent	silent	silent	stop	detected	silent
	swap_rand	stop	detected	detected	stop	detected	silent

TABLE IV: The summary of application behaviors facing various metadata corruptions. *It includes results both with and without LFSCCK run.*

normally, which results in silent corruption. It is clear that how careful the application developers are about data access leads to different symptoms.

2) The LOVEA random case leads to *stop* for Nyx, *silent* for Montage, and *detected* for 1000Genome. Note that, when the LOVEA is *random*, the corrupted file becomes entirely inaccessible as the random LOVEA will point to non-exist OSS objects. When such a corruption happens, Nyx stops the execution as it is unable to open the inaccessible file. It raises a run time error and stops execution. Montage, on the other hand, ignores the inaccessible file without reporting anything. It continues to run and generate wrong output that do not match the expected results (silent corruptions). For 1000Genome, when the file is inaccessible, it reports `File Not Found` error but continues processing the remaining files. Both Montage and 1000Genome produce outputs that do not match the expected results.

3) The LOVEA *swap_similar* case leads to *silent* for all applications regardless how the applications check their data files. Note that, *swap_similar* means the LOVEA of a corrupted file is swapped with the LOVEA of another similar file. Similar file means the file contains meaningful data but does not belong to the input dataset of the current execution. In this case, none of the three applications report any issues, as the file is accessible and appears to meet the expected format. However, as the content does not match the original input file, the generated output does not match the expected results. The corruption is silent in all three applications. Such a consequence is the most challenging one to deal with as additional checking inside the application will still not work. But the consequence (silent error) could be catastrophic. It is clear that additional system-level mechanism is necessary to handle such a case.

4) The LOVEA *swap_random* case leads to *stop* for Nyx, *detected* for Montage and 1000Genome. This indicates the case where the corrupted file mimics the content of a random file (*swap_rand*), which does not have the expected format as per the applications. Both Nyx and Montage detects the corruptions: Nyx stops, while Montage reports the corrupted file and continues processing the remaining input files. 1000Genome does not explicitly verify the file format, but the random content does not match the expected

structure of a 1000Genome input file. While processing the corrupted file, 1000Genome encounters a runtime error due to this mismatch. The error is reported for the corrupted file, and 1000Genome continues processing the remaining files, producing incorrect output.

From above analysis, we can observe that applications show different symptoms facing metadata corruptions. This is clearly relevant with how applications verify their input files, which further raise the question that if better programming practice can completely handle metadata corruptions. In the next subsection, we will further analyze previous behaviors in the source code level and discuss if application developers can easily handle metadata corruptions from their ends.

D. Application Behaviors from Source Code

Based on the previous analysis, we identified two main factors that determine their behavior: 1) how the application accesses the files and 2) the verification it performs on the file after access.

We present two cases that helps illustrate this better: 1) when LOVEA is corrupted with a random value and 2) when LOVEA is missing. In both cases, all three applications shows different behaviors in response to the same fault.

```

nyx/receiver/.../amt-connected-components.cpp
1 int main(input_file...)
2   read_from_file(input_file);
3   if file_exists(input_file);
4   process file;
5   else
6     throw runtime_error("Cannot read file" + input_file);
7 return;

a) Nyx

montage/get_files.c
1 void get_files(input_directory)
2   opendir(input_directory)
3   while (entry = readdir(input_directory))
4     if (stat(entry))
5       if (S_ISDIR(entry))
6         process directory;
7       else
8         process file;
9   closedir(input_directory);
10 return;

b) Montage

1000genome/bin/individuals.py
1 processing(input_file,...)
2   with open(output_file, 'r') as f:
3     content = f.readlines()
4     process input_file content;
5 return;

c) 1000Genome

```

Fig. 4: Analysis for LOVEA random case

Fig. 4 shows the code of all three applications reacting to the LOVEA random faults. In this case, the corrupted file becomes inaccessible. Nyx opens the input file by specifically using the file name, and verifies its accessibility (Fig. 4(a), line 3). Upon failing to open the file, it throws a runtime

exception and terminates execution (i.e., stop). In contrast, Montage (Fig. 4(b), line 3) accesses its input files differently. Instead of explicitly opening the input files, it reads them one by one from a directory. Then, it performs a 'stat()' operation to retrieve file attributes before accessing each file (line 4). However, when 'stat()' fails, Montage does not have a mechanism to handle the failure. As a result, it simply ignores the corrupted file and moves on to the next input file, leading to silent errors. 1000Genome, similar to Nyx, explicitly opens the input file by its name (Fig. 4.c, line 2). Since 1000Genome uses Python to open files. Python automatically handles the failure by raising a File Not Found error. However, since 1000Genome processes ten chromosome files independent of each other and the corruption only affects the *chromosome-1* file, it continues to process the remaining files.

```

nyx/reader/./hdf5-reader.cpp
1 void read_from_hdf5_file(input_file, keyword);
2 if(input_file.getDataSet(keyword))
3     process file;
4 else
5     throw runtime_error("Cannot read file" + input_file);
6 return;

a) Nyx

montage/get_hdr.c
1 int get_hdr(file)
2 { if(fits_open_file(file) != 0)
3     badfile++;
4     continue processing next file;
5 else
6     process file;
7 return;

b) Montage

1000genome/bin/individuals.py
1 processing(input_file,...)
2 with open(output_file, 'w') as f:
3     for line in input_file:
4         process input_file;
5 return;

c) 1000Genome

```

Fig. 5: Analysis for LOVEA missing case

Fig. 5 shows the code of three applications reacting to LOVEA missing case. In this case, the corrupted file is accessible but empty due to the corruption. After successfully accessing the file, Nyx performs verification checks to ensure the file has the expected format. It does this by checking for specific keywords (Fig. 5.a, line 2). If the file fails the verification, Nyx throws a runtime error and terminates. Similarly, Montage performs file verification once the file is accessed. The 'fits_open_file()' method called by Montage (Fig. 5.b, line 2) verifies the file's content for expected keywords. If the file fails verification, Montage records it as "badfits" (bad .fits file) and moves on to the next file. In contrast, 1000Genome does not perform any verification to ensure the file is correct after accessing it. Since the input file is empty due to corruption, the for loop (Fig. 5.c, line 3), which reads the input file line by line, processes it, and populates the output file, will fail execution. As a result, the output file remains empty without triggering any alert.

From these examples, it is evident that thoroughly checking the data file within the data access code can help detect metadata corruptions. For instance, Nyx identifies the LOVEA missing case and halts execution by checking a keyword after reading the data file. However, this approach cannot serve as a general solution for several reasons. First, the method by which an application accesses its data can vary significantly: some may know the exact file name, while others may need to access all files within a directory. It is impractical to verify the existence of every file in advance in all

cases. Second, checking the actual contents of a file presents challenges: 1) similar files may exist, and 2) the file contents may lack a specific keyword to check. While more careful programming practices would undoubtedly help minimize the catastrophic (often silent) impacts of metadata corruptions on applications, this approach alone is insufficient. In the next subsection, we explore another option: leveraging the file system checking mechanisms, which are widely available in parallel file systems, to identify and address these issues.

E. Application Behavior after LFSCK

We repeated the same experiment, but this time, we executed LFSCK first. LFSCK is a Lustre file system checker designed to detect and repair Lustre metadata corruptions. We expected LFSCK to successfully identify and repair the metadata corruptions injected by our fault model, enabling the applications to function correctly after the LFSCK run.

However, results in Table IV does not support such a claim. We can observe that application behaviors do change after the LFSCK run. In some cases, the application generates correct output due to the LFSCK repair, such as LMA random cases. However, in most cases, the applications continue to experience partial failures, implying that LFSCK is unable to restore the corrupted metadata correctly.

For instance, in the case of LMA missing, swap_similar and swap_rand, LFSCK is unable to repair LMA and restore it to its original value. In case of missing LMA, LFSCK created a new file object to replace the corrupted object in the file system namespace. A new LMA is generated for this file, along with new LOVEA and data objects, resulting in an empty file. When the LMA is swapped with that of a similar file or a random file, LFSCK does not repair the corrupted LMA. Instead, it repairs the file's LOVEA to match that of the duplicated file, causing the corrupted file to mimic the content of the duplicated file. The three applications behave similarly to the scenarios where the file was either empty or mimicking the content of a similar or random file, as discussed previously. When the LMA is corrupted with a random value, LFSCK is able to restore it to its original value. This is the only scenario where LFSCK is capable of handling this type of corruption and repairing it correctly, resulting in all three applications generating correct output.

In the case of LOVEA corruptions, specifically, the missing and random fault models, LFSCK is unable to detect the corruptions and does not perform any repairs. When LOVEA is missing, the corrupted file remains accessible but empty, and when LOVEA is corrupted with a random value, the file remains inaccessible. The applications behave the same way as they do without running LFSCK. When LOVEA is corrupted with swap_similar or swap_rand fault model, LFSCK repairs the LOVEA by creating new data objects for the file and updating the LOVEA accordingly. As a result, the repaired file is empty, and the applications behave the same way as they did when the corrupted file was empty.

From this analysis, we conclude that LFSCK is unable to correctly detect and repair these faults, and therefore cannot

mitigate the adverse effects on application behavior. It is clear a new mechanism is needed.

IV. METADATA CHECKSUM

In the previous section, we observed that metadata corruption negatively impacts applications, potentially leading to silent and catastrophic errors. Since applications are not designed to manage filesystem corruption, they rely on the filesystem to work correctly. Although LFSCK is expected to be able to handle such corruptions, it too, cannot detect or recover the corrupted metadata for many cases.

In this study, we propose leveraging a simple runtime metadata checksum mechanism to address this issue. Since the underlying local-level file system, Ext4-based *ldisks*, already supports a metadata checksum feature [2], the most intuitive solution would be to enable it for Lustre. However, upon further investigation and discussions with the Lustre development team, we discovered that Lustre does not support the Ext4 metadata checksum mechanism due to an incompatibility issue. Specifically, this incompatibility arises because the Ext4 metadata checksum feature conflicts with another Lustre feature—the extension of directory name entries to include Lustre metadata [43]. For this reason, we decided to implement and test our own FUSE-based checksum mechanism to rapidly validate our approach. If successful, we plan to investigate in addressing the incompatibility issue and integrate this mechanism directly into Lustre in the future.

We utilize the FUSE interface [5] to integrate our checksum mechanism with the Lustre filesystem and run the applications on top of this configuration. While FUSE introduces significant performance overhead, our primary objective is to evaluate the checksum mechanism's capability to detect metadata corruptions. To achieve better performance, the checksum mechanism can later be integrated directly into the filesystem used by the application. Currently, our checksum mechanism stops the application from further execution upon detecting corruption.

A. Fuse-Based Metadata Checksum

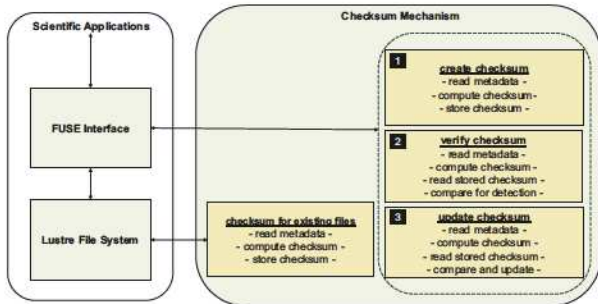


Fig. 6: Fuse-based Metadata Checksum workflow

Our checksum workflow is illustrated in Fig. 6. Three primary components, *create checksum*, *verify*

checksum, and *update checksum*, are implemented using the FUSE interface. Since we apply our checksum tool to a pre-existing filesystem, we introduce an additional component, *checksum for existing files*. This component generates checksums for all application files that already exist before the application execution begins. After this step, we run the applications utilizing the three FUSE-based checksum components.

- *create checksum*: Generates a checksum for every new file created by the application during execution, such as intermediate and output files.
- *verify checksum*: Verifies the correctness of a file's metadata whenever the application accesses the file.
- *update checksum*: Updates the checksum to reflect any changes made to the file's metadata by the application.

The details of each component are as follows:

1) *checksum for existing files*: It consists of three steps: a) **read metadata**: We target the Lustre extended attributes (LMA and LOVEA), which are essential for file access. The `get_xattr()` method is used to read these attributes.

b) **compute checksum**: The attribute values obtained in the previous step are concatenated, and a CRC32C checksum is computed. The CRC32C checksum method is identical to that used by the Ext4 filesystem for computing its metadata checksums [2]. Our CRC32C implementation is software-based, utilizing the `zlib` library.

c) **store checksum**: A new user-type extended attribute named `csum` is created in the file's inode to store the computed checksum. Lustre allows creating user-type extended attributes when the `user_xattr` option is set during mounting [7]. We use the `set_xattr()` method [8] to create the `csum` attribute.

2) *create checksum*: When the application creates a new file, it utilizes the `mknod()` operation. We intercept this operation via the FUSE interface to introduce our *create checksum* component. The same three steps described above, namely a) **read metadata**, b) **compute checksum** and c) **store checksum**, are performed, assuming that the newly created file has no existing metadata corruption.

3) *verify checksum*: When the application accesses files, we intercept file-access operations (such as `'getxattr()'`, `'lstat()'`, `'open()'` and `'read()'`) via the FUSE interface to verify metadata correctness. The component consists of four steps: The first two steps are identical to those used by the previous components, that is a) **read metadata** and b) **compute checksum**. The third steps c) **read stored checksum** is different: we use the `'getxattr()'` method to read the `csum` extended attribute, which contains the previously computed checksum for verification. The last step d) **compare checksums** is to detect corruption. If the computed checksum matches the stored `csum`, the metadata is considered corruption-free, and the intercepted operation continues normally. If the checksums do not match, indicating the presence of metadata corruption, an interrupt is

Metadata Field	Fault Model	Checksum Verification	Nyx	Montage	1000Genome
LMA	missing	detected corruption*	stop*	stop*	stop*
	random	detected corruption-	stop-	stop-	stop-
	swap_similar	detected corruption-	stop-	stop-	stop-
	swap_rand	detected corruption-	stop-	stop-	stop-
LOVEA	missing	detected corruption*	stop*	stop*	stop*
	random	detected corruption	stop	stop	stop
	swap_similar	detected corruption	stop	stop	stop
	swap_rand	detected corruption	stop	stop	stop

TABLE V: Checksum Functionality Evaluation. * means detects corruption and stops because `getxattr()` fails due to the missing `xattr`; - means detects corruption and stops because `getxattr()` can't access the `xattr` due to corruption.

triggered to stop the application execution. Note that this mechanism only detects the presence of corruption and stops execution; it neither identifies the specific corrupted metadata fields nor attempts to repair the corruption.

4) *update checksum*: We intercept file operations that modify Lustre file metadata, particularly `'setxattr()'`, `'truncate()'`, and `'write()'`, using the FUSE interface. Although not every `write()` operation changes metadata, creating new OST data objects does modify it. Additionally, we intercept the `close()` operation to ensure the stored checksum remains up-to-date before closing the file. This component consists of four steps, the first three of which resemble those used by the `verify checksum` component: a) **read metadata**, b) **compute checksum** and c) **read stored checksum**. The final step d) **compare and update the checksum** is different: we compare the computed checksum with the stored `csum`. If they differ, we update `csum` with the newly computed checksum.

V. EVALUATIONS OF METADATA CHECKSUM

In this section, we evaluate whether our checksum mechanism can effectively detect metadata corruption and prevent silent failures. Additionally, we assess the performance overhead introduced by our FUSE-based implementation. As previously discussed, the FUSE layer itself may cause performance overheads; however, our primary focus is specifically on the additional overhead caused by the checksum mechanism itself. This overhead measurement can serve as an indicator of the expected performance impact if the checksum mechanism is later integrated directly into Lustre.

Following this approach, our evaluation focuses on two aspects: 1) *functionality*: Whether the checksum tool effectively detects metadata corruption according to our fault model and halts application execution, thereby preventing silent failures. 2) *performance*: The performance overhead introduced specifically by the checksum mechanism.

All evaluations are performed on a Lustre filesystem deployed on CloudLab [1]. The Lustre setup consists of one combined MGS/MDS server and four OSS servers, configured with default striping. Each server is equipped with an Intel® Xeon® E5-2660 CPU, 157 GB of RAM, and a 1 TB local disk. The Lustre version used is 2.12.5, providing a total storage capacity of 2 TB. The Lustre client utilizes the FUSE interface (version 2.9.2). Additionally, Lustre is

mounted with the `user_xattr` option enabled, and the `oi-scrub` feature is disabled. This configuration ensures that our checksum mechanism is evaluated independently from Lustre's built-in consistency checker (LFSCCK).

A. Metadata Checksum-Functionality Evaluation

Similar to Section III.A., we select one input file per application to inject each corruption from our fault model in both the LMA and LOVEA metadata of the file inode. We then observe the impact of the checksum mechanism on the application's behavior and record it in Table V. From these results, we can quickly conclude that the checksum mechanism successfully detects the metadata corruption and stops the application from executing further.

In some cases, we are unable to read the LMA or LOVEA attribute for checksum computation due to the injected faults. For example, when LOVEA is `missing`, the `'getxattr()'` method fails to read the attribute and returns an error. The checksum mechanism still successfully handles such cases by identifying the returned error.

B. Checksum Performance Overhead Evaluation

To evaluate the performance of the checksum mechanism, we do two types of analysis. First, we conduct an overall runtime comparison to measure the overhead introduced by the checksum. Next, we do a detailed analysis to break down the time required for each step of the checksum mechanism.

1) *Runtime Comparison*: We calculate the time required to complete application execution in two different modes: a) $T_{App+FUSE}$: application runtime with the FUSE interface and no checksum mechanism, and b) $T_{App+FUSE+Csum}$: application runtime with FUSE interface and the checksum mechanism. We do not introduce corruption in these scenarios, as our goal is to allow each application to complete a full run. For accuracy, we run each application three times under each condition, calculate the average execution time, and report in Table VI. From these results, we can see that all three applications have minimal overheads due to the additional checksum mechanism.

Note here, Nyx uses 'HDF5' format [42] to access the data. Its input file is 6.6 GB in size, stored in binary format, and processed in small chunks. The application opens, processes, and closes each file multiple times, leading to numerous

Application	$T_{App+Fuse}$	$T_{App+Fuse+Csum}$
Nyx	13 sec	13.10 sec
Montage	34.35 sec	34.37 sec
1000Genome	51 min 54 sec	52 min 3 sec

TABLE VI: Checksum Performance Evaluation

number of metadata operations and requiring checksum operations. Montage uses the 'fits' file format [36], which is simpler, with a header followed by sections of raw data. It has input dataset of ten files, each 2.1 MB in size, containing binary-encoded data. 1000Genome uses the simplest data formats. Each file is 2.8 GB in size, it is stored in plain text format, making it easier to process.

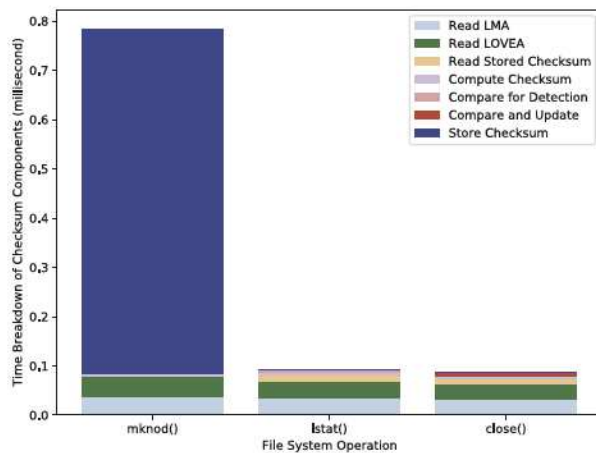


Fig. 7: Representative file operations in the checksum model: 'mknod()' for creation, 'lstat()' for verification, and 'close()' for update

2) *Breakdown Analysis*: Fig. 7 shows the detailed breakdown of the time required by each step of the three components of the checksum mechanism: create checksum, verify checksum, and update checksum. The 'mknod()', 'lstat()' and 'close()' operations are used to report time as representative of the three components. We have measured the time for each operation five times and reported the average. The time for each operation is in milliseconds. Each step of the three components are represented, with different colours. The read metadata step is further broken down to read LMA and read LOVEA.

From the plot, we can see that the store checksum step of the 'mknod()' operation takes the most time compared to all other steps as it is responsible for creating an extended attribute on disk and storing the checksum in it. However, once the attribute is created, updating it becomes significantly faster as can be seen from the compare and update step of the 'close()' operation. The read LMA and read LOVEA steps introduces some delay compared to the read stored checksum step. All three steps reads an extended attribute from disk. But the checksum data is close to the original metadata and benefit the reading performance as it

is likely already loaded in memory. In addition, the checksum data size is small (4 bytes) compared to the size of the LMA and LOVEA attributes which were 24 bytes for LMA, and 56 bytes for LOVEA for default stripe setting, further increasing the chance of being pre-loaded. Once the attributes are read from the disk they are processed in-memory, thus significantly reducing time for steps that calculates the checksum and compares the computed and stored checksum, that is compute checksum, compare for detection and compare and update steps.

VI. RELATED WORK

1) *PFS fault analysis and checkers*: In the field of high-performance computing, many studies have focused on PFS fault analysis and the evaluation of PFS checkers [17, 18, 20, 28, 33]. Cao et. al. [18] propose a framework called PFault, designed to test the fault tolerance of parallel file systems by emulating failures such as whole device failures, global and local file system inconsistencies, and network partitioning. The framework then invokes the file system checker and assesses its ability to recover the PFS. Similarly, Cao et. al. [17] propose a generic framework for testing failure handling. This framework captures I/O commands from dedicated workloads and manipulates them to emulate real-world failure scenarios. However, these studies focus on general PFS faults and do not specifically analyze faults in PFS metadata. Dai et. al. [20] analyze the functionality and performance of the PFS checker. Han et. al. [28] study the defects in PFS checker policies through fault injection. However, neither study analyzes the impact of these faults on the broader file system mechanisms or the applications utilizing the PFS.

2) *Analyzing partial failures in HPC systems*: Many studies have analyzed partial failures [12, 13, 23, 35]. For instance, Bairavasundaram et. al. [12] conducted an extensive analysis to understand the impact of data corruption in storage systems, particularly focusing on silent data corruptions such as checksum mismatches, parity inconsistencies, identity discrepancies, and data loss. Their study spanned 41 months and involved examining over 1.53 million disk drives, offering valuable insights into the characteristics of these corruptions. Similarly, Narayan et. al. [35] investigate silent data corruption on disk drives resulting from misdirected writes, torn writes, data path corruption, bit-rot, and firmware bugs. They further developed a CRC check tool to address these issues and analyzed its effectiveness. Fang et. al. [23] characterized the behavior of scientific applications in response to data faults in SSD devices by building the FFIS-a FUSE-based fault model, which simulates data corruptions such as bit-flip, shorn write, and dropped write to assess applications' error resilience. However, these studies primarily focus on partial failures caused by data corruption, while our work analyzes failures caused by metadata corruption.

VII. CONCLUSION AND FUTURE PLAN

In this study, we examined the impact of metadata corruption on parallel file systems. Specifically, we designed a

fault model targeting metadata essential for file access on the Lustre filesystem and analyzed the behavior of three scientific applications: Nyx, Montage, and 1000Genome. Our analysis revealed that metadata corruptions could cause both obvious errors and catastrophic silent failures in application outputs. Additionally, we observed that Lustre's built-in consistency checker (LFSCK) was insufficient in detecting and handling such corruptions. By examining the source code of the applications, we also determined that while factors such as file-access mechanisms and internal verification methods can contribute to error detection, they alone are inadequate for fully mitigating issues arising from metadata corruption. To address these challenges, we proposed a checksum mechanism capable of detecting metadata corruption and halting application execution to prevent silent errors. Our evaluation demonstrated that this checksum tool effectively detects all corruption cases defined in our fault model. Furthermore, assessments of the tool's impact on application runtime and analysis of the execution time required by each component showed that the checksum mechanism itself introduces minimal overhead and is time-efficient.

In future work, we first plan to integrate our checksum mechanism directly into Lustre and comprehensively evaluate its performance. In addition, we aim to expand our fault model to include corruptions in directory metadata and data objects stored on OSS servers.

VIII. ACKNOWLEDGMENTS

We sincerely thank the anonymous reviewers for their valuable feedback. This work was supported in part by National Science Foundation (NSF) under grants CNS-2008265, CCF-2412345, CCF1910747, CNS1943204, CNS2402858, and a Global Research Outreach (GRO) Award from Samsung Advanced Institute of Technology (SAIT) and Samsung Research America (SRA). This effort was also supported in part by the U.S. Department of Energy (DOE) through the Office of Advanced Scientific Computing Research's "Orchestration for Distributed & Data-Intensive Scientific Exploration" and the "Cloud, HPC, and Edge for Science and Security" LDRD at Pacific Northwest National Laboratory. Pacific Northwest National Laboratory is operated by Battelle for the DOE under Contract DE-AC05-76RL01830.

REFERENCES

- [1] CloudLab. <https://www.cloudlab.us/>. Accessed: 2024-10-10.
- [2] Ext4 Metadata Checksums. https://ext4.wiki.kernel.org/index.php/Ext4_Metadata_Checksums. Accessed: 2024-08-04.
- [3] LFSCK: An Online File System Checker for Lustre. <https://github.com/Xyratex/lustre-stable/blob/master/Documentation/lfscck.txt>. Accessed: 2024-08-14.
- [4] LFSCK Phase 1 - OI Scrub Solution Architecture. https://wiki.lustre.org/LFSCK_Phase_1_-_OI_Scrub_Solution_Architecture. Accessed: 2024-10-4.
- [5] libFUSE: The Reference Implementation of the Linux FUSE (Filesystem in Userspace) Interface. <https://github.com/libfuse/libfuse>. Accessed: 2024-08-14.
- [6] Lustre File System. <http://opendsfs.org/lustre/>. Accessed: 2024-08-14.
- [7] Mounting a Lustre File System on Client Nodes. https://wiki.lustre.org/Mounting_a_Lustre_File_System_on_Client_Nodes. Accessed: 2024-10-4.
- [8] setattr(2) - Linux Manual Page. <https://man7.org/linux/man-pages/man2/setattr.2.html>. Accessed: 2024-10-8.
- [9] Top500 Supercomputers: NOVEMBER 2018. <https://www.top500.org/lists/2018/11/>. Accessed: 2024-08-14.
- [10] HPC Power Outage Event at Texas Tech. <http://www.ece.iastate.edu/~mai/docs/failures/2016-hpcc-lustre.pdf>, 2016.
- [11] Ann S. Almgren, John B. Bell, Mike J. Lijewski, Zarija Lukić, and Ethan Van Andel. Nyx: A Massively Parallel AMR Code for Computational Cosmology. *The Astrophysical Journal*, 2013.
- [12] Lakshmi N. Bairavasundaram, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Garth R. Goodson, and Bianca Schroeder. An Analysis of Data Corruption in the Storage Stack. *ACM Transactions on Storage (TOS'08)*.
- [13] Lakshmi N. Bairavasundaram, Garth R. Goodson, Shankar Pasupathy, and Jiri Schindler. An Analysis of Latent Sector Errors in Disk Drives. In *Proceedings of the 2007 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS'07)*.
- [14] Hanmant P. Belgal, Nick Righos, Ivan Kalastirsky, Jeff I. Peterson, Robert Shiner, and Neal Mielke. A New Reliability Model for Post-Cycling Charge Retention of Flash Memories. In *Proceedings of 2002 IEEE International Reliability Physics Symposium (IRPS'02)*.
- [15] Simona Boboila and Peter Desnoyers. Write Endurance in Flash Drives: Measurements and Analysis. In *Proceedings of the 8th USENIX Conference on File and Storage Technologies (FAST '10)*.
- [16] Yu Cai, Erich F. Haratsch, Onur Mutlu, and Ken Mai. Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and Analysis. In *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE'12)*.
- [17] Jinrui Cao, Simeng Wang, Dong Dai, Mai Zheng, and Yong Chen. A Generic Framework for Testing Parallel File Systems. In *Proceedings of 1st Joint International Workshop on Parallel Data Storage and Data Intensive Scalable Computing Systems (PDSW-DISCS'16)*.
- [18] Jinrui Cao, Simeng Wang, Dong Dai, Mai Zheng, and Yong Chen. PFault: A General Framework for Analyzing the Reliability of High-Performance Parallel File Systems. In *Proceedings of the 32nd ACM International Conference on Supercomputing (ICS'18)*.
- [19] Philip H. Carns, Walter B. Ligon III, Robert B. Ross, and Rajeev Thakur. PVFS: A Parallel File System for Linux Clusters. In *4th Annual Linux Showcase & Conference (ALS'00)*.
- [20] Dong Dai, Om Rameshwar Gatla, and Mai Zheng. A Performance Study of Lustre File System Checker:

- Bottlenecks and Potentials. In *2019 35th Symposium on Mass Storage Systems and Technologies (MSST'19)*.
- [21] Dong Dai, Chen Yong, Philip Carns, John Jenkins, Zhang Wei, and Robert Ross. GraphMeta: A Graph-Based Engine for Managing Large-Scale HPC Rich Metadata. In *2016 IEEE International Conference on Cluster Computing (CLUSTER'16)*.
 - [22] Chris Egersdoerfer, Di Zhang, and Dong Dai. Early Exploration of Using ChatGPT for Log-Based Anomaly Detection on Parallel File Systems Logs. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC'23)*.
 - [23] Bo Fang, Daoce Wang, Sian Jin, Quincey Koziol, Zhao Zhang, Qiang Guan, Suren Byna, Sriram Krishnamoorthy, and Dingwen Tao. Characterizing Impacts of Storage Faults on HPC Applications: A Methodology and Insights. In *2021 IEEE International Conference on Cluster Computing (CLUSTER'21)*.
 - [24] Aishwarya Ganesan, Ramnathan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Redundancy Does Not Imply Fault Tolerance: Analysis of Distributed storage Reactions to File-System Faults. *ACM Transactions on Storage (TOS'17)*.
 - [25] Om Rameshwar Gatla, Muhammad Hameed, Mai Zheng, Viacheslav Dubeyko, Adam Manzanares, Filip Blagojević, Cyril Guyot, and Robert Mateescu. Towards Robust File System Checkers. In *16th USENIX Conference on File and Storage Technologies (FAST'18)*.
 - [26] Haryadi S. Gunawi, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. SQCK: A Declarative File System Checker. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI'08)*.
 - [27] Runzhou Han, Om Rameshwar Gatla, Mai Zheng, Jinrui Cao, Di Zhang, Dong Dai, Yong Chen, and Jonathan Cook. A Study of Failure Recovery and Logging of High-Performance Parallel File Systems. *ACM Transactions on Storage (TOS'22)*.
 - [28] Runzhou Han, Duo Zhang, and Mai Zheng. Fingerprinting the Checker Policies of Parallel File Systems. In *2020 IEEE/ACM Fifth International Parallel Data Systems Workshop (PDSW'20)*.
 - [29] Jan Heichler. An introduction to BeeGFS. https://www.beegfs.de/docs/whitepapers/Introduction_to_BeeGFS_by_ThinkParQ.pdf. Accessed: 2024-08-14.
 - [30] Pegasus ISI. 1000Genomes Workflow. <https://github.com/pegasus-isi/1000genome-workflow>. Accessed: 2024-08-04.
 - [31] Joseph C. Jacob, Daniel S. Katz, G. Bruce Berri-man, John C. Good, Anastasia Laity, Ewa Deelman, Carl Kesselman, Gurmeet Singh, Mei-Hui Su, Thomas Prince, et al. Montage: A Grid Portal and Software Toolkit for Science-Grade Astronomical Image Mosaicking. *International Journal of Computational Science and Engineering (IJCE'09)*.
 - [32] Shehbaz Jaffer, Stathis Maneas, Andy Hwang, and Bianca Schroeder. Evaluating File System Reliability on Solid State Drives. In *2019 USENIX Annual Technical Conference (ATC'19)*.
 - [33] Saisha Kamat, Abdullah Al Raqibul Islam, Mai Zheng, and Dong Dai. FaultyRank: A Graph-Based Parallel File System Checker. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS'23)*.
 - [34] Tabassum Mahmud, Om Rameshwar Gatla, Duo Zhang, Carson Love, Ryan Bumann, and Mai Zheng. ConfD: Analyzing Configuration Dependencies of File Systems for Fun and Profit. In *21st USENIX Conference on File and Storage Technologies (FAST'23)*.
 - [35] Sumit Narayan, John A. Chandy, Samuel Lang, Philip Carns, and Robert Ross. Uncovering Errors: The Cost of Detecting Silent Data Corruption. In *Proceedings of the 4th Annual Workshop on Petascale Data Storage (PDSW'09)*.
 - [36] NASA. A Beginner's Guide to the FITS Format. https://fits.gsfc.nasa.gov/fits_primer.html. Accessed: 2024-10-10.
 - [37] NERSC. Nyx High-Redshift Simulations. <https://portal.nersc.gov/project/nyx/highz/512/>. Accessed: 2024-08-04.
 - [38] OpenSFS and Whamcloud. Lustre software release 2.x operations manual. https://doc.lustre.org/lustre_manual.xhtml. Accessed: 2024-08-14.
 - [39] Vijayan Prabhakaran, Lakshmi N. Bairavasundaram, Nitin Agrawal, Haryadi S. Gunawi, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. IRON File Systems. In *Proceedings of the 20th ACM Symposium on Operating Systems Principles (SOSP'05)*.
 - [40] Paul Randal. PFS Corruption After Upgrading from SQL Server 2014. <https://www.sqlskills.com/blogs/paul/pfs-corruption-after-upgrading-from-sql-server-2014/>. Accessed: 2024-08-14.
 - [41] Jingjing Ren, Ashwin Rao, Martina Lindorfer, Arnaud Legout, and David Choffnes. Recon: Revealing and Controlling PII Leaks in Mobile Network Traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys'16)*.
 - [42] The HDF Group. Hdf5 file format specification. <https://docs.hdfgroup.org/archive/support/HDF5/doc/H5.format.html>. Accessed: 2024-10-10.
 - [43] Whamcloud. Lustre 2.13 Client Can't Open Files Larger Than 8TB with "No Space Left on Device" Error. <https://jira.whamcloud.com/browse/LU-13650>. Accessed: 2024-08-14.
 - [44] Erci Xu, Mai Zheng, Feng Qin, Yikang Xu, and Jiesheng Wu. Lessons and Actions: What We Learned from 10K SSD-Related Storage System Failures. In *Proceedings of the 2019 USENIX Conference on USENIX Annual Technical Conference (ATC'19)*.
 - [45] Di Zhang, Dong Dai, Runzhou Han, and Mai Zheng. Sentilog: Anomaly Detecting on Parallel File Systems

- via Log-Based Sentiment Analysis. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage'21)*.
- [46] Di Zhang, Chris Egersdoerfer, Tabassum Mahmud, Mai Zheng, and Dong Dai. Drill: Log-Based Anomaly Detection for Large-Scale Storage Systems Using Source Code Analysis. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS'23)*.
- [47] Mai Zheng, Joseph Tucek, Feng Qin, and Mark Lillibridge. Understanding the Robustness of SSDS under Power Fault. In *Proceedings of the 11th USENIX Conference on File and Storage Technologies (FAST'13)*.